

Dokumentacja jako graf

Piszesz treść. Program wie, co z czym się łączy, i odpowiada na trzy pytania, na które dziś odpowiada przeszukiwanie folderów.

ZMIANA

Które pliki muszę przejrzeć, gdy zmieniam to wymaganie?

`change_scope`

BUDOWA

Co muszę przeczytać, zanim zbuduję ten scenariusz?

`implementation`

JAKOŚĆ

Czy dokumentacja jest spójna i kompletna?

3 walidatory · 17 sprawdzeń

Dlaczego warto



Jedno pole `relations`

Wyliczane z treści przez skrypt. Pisziesz dokument, nie listę powiązań.



Relację zapisuje tylko źródło

Listę „kto mnie wskazuje” liczy program — nic się nie rozjeżdża.



Zasięg zmiany jednym zapytaniem

Odpowiedź w milisekundach: lista plików do przejrzenia, z drogą w grafie.



Błąd wychodzi przy zapisie

Plik, linia, kod i podpowiedź naprawy — zanim ktoś trafi na błąd w implementacji.

14

plików do przejrzenia przy zmianie `BFS-001.REQ-01` — zamiast całego folderu

4,4×

szybciej agent AI znajduje pliki z indeksem

36/36

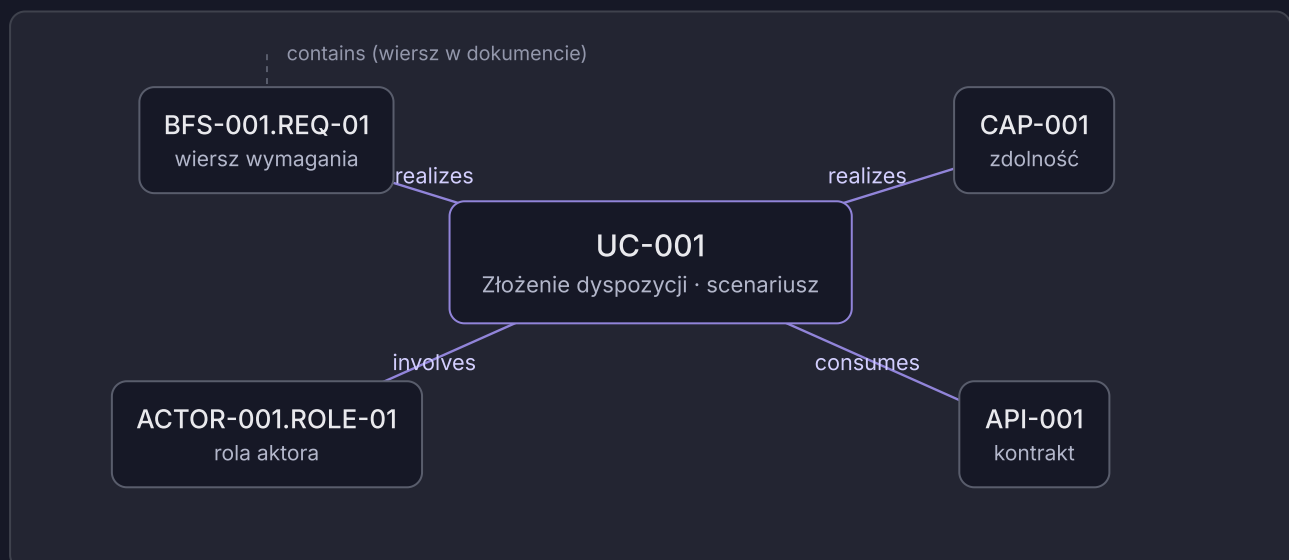
bez błędnych odpowiedzi agenta z indeksem; bez indeksu 19/36

0

pakietów npm do instalacji; wystarczy `Node.js`

Jak to działa: trzy pojęcia

Węzeł, krawędź i jedna reguła. Resztę liczy program.



Relacje scenariusza UC-001. Strzałka wychodzi zawsze ze źródła; BFS-001 nie ma wpisu w drugą stronę.

WĘZEL

Dokument albo jego wiersz

UC-001 to dokument. BFS-001.REQ-01 to wiersz wymagania z własnym ID.

KRAWĘDŹ

Wpis w polu `relations`

Rodzaj i cel, np. `realizes CAP-001`. Rodzaju nie wybierasz — dla pary typów matryca podaje dokładnie jeden.

JEDNA REGUŁA

Relacja wynika z treści

Jeśli dokument wspomina identyfikator, ma do niego wpis. Wpis dopisuje skrypt, nie człowiek.

```
doc-id: UC-001
title: Złożenie dyspozycji
status: draft
relations:
  - type: realizes
    target: BFS-001.REQ-01
  - type: realizes
    target: CAP-001
  - type: involves
    target: ACTOR-001.ROLE-01
  - type: consumes
    target: API-001
```

STANDARD W LICZBACH

21

typów dokumentów,
każdy z szablonem i
instrukcją

6

rodzajów relacji:
`contains`, `realizes`,
`constrained_by`,
`consumes`, `involves`,
`groups`

68

dozwolonych trójek w
matrycy

10

typów wierszy: `REQ`, `RB`,
`DOM`, `RD`, `RW`, `NFRT`,
`ROLE`, `AC`, `A`, `E`

Od treści do odpowiedzi

Jedna ścieżka danych. Analitik pracuje tylko na pierwszym kroku.

1 · ANALITYK

Piszesz treść

Z szablonu. Wpisujesz identyfikatory, od których dokument zależy.

VS Code

2 · SKRYPT

Relacje z treści

Znajduje identyfikatory, bierze rodzaj z matrycy, zapisuje `relations`.

```
sync-relations --fix
```

3 · SKRYPTY

Walidacja

Graf, forma, spójność. Plik, linia, kod i odpowiedź naprawy.

REL · FORM · CONS

4 · INDEKS

Odpowiedzi

Zasięg zmiany i lista do przeczytania — w terminalu, aplikacji i u agenta AI.

CLI · MCP · Warsztat

Skrypt wylicza wpisy z tego, co napisałeś

W TREŚCI		WPIS W RELATIONS
UC-001 wspomina CAP-001	→	realizes CAP-001
BFS-001 wspomina własny wiersz REQ-01	→	contains REQ-01
API wspomina ENT-Disposition.RW-001, a API nie może wskazać reguły RW	→	consumes ENT-Disposition
UC wspomina REQ-01 bez prefiksu dokumentu	→	RĘCZNIE popraw treść

Walidator mówi co, gdzie i jak naprawić

```
docs/processes/BFS-001/contracts/int-002.md:20
  BŁĄD CONS-01 ta sama informacja w dwóch
  miejscach
  INT-002 ma wpis realizes CAP-001,
  a pole „Zdolność” go nie wymienia.
  Naprawa: Dopisz CAP-001 w polu „Zdolność”.
```

REL 01-08

graf: cel istnieje, liczebność, unikalny doc-id, brak wysp, numery nie wracają

FORM 01-06

szablon: pola, sekcje, tabele, nazwa pliku, folder

CONS 01-03

spójność: relacja vs. treść, progi NFR, krok ma ekran i rolę

Klik w VS Code przenosi do linii. `--json` dla programów, kod wyjścia 1 dla CI.

Czy warto? Test z agentem AI

6 zadań · 3 modele · 72 uruchomienia. To samo pytanie z indeksem (MCP) i bez. Wzorzec: lista plików z indeksu.



bez indeksu (Read, Grep, Glob)

z indeksem knowledge-index (MCP)

Trafność według modelu

	TRAFIONE PLIKI		KOSZT
Haiku bez		75%	0,171 \$
Haiku z MCP		100%	0,041 \$
Sonnet bez		96%	0,290 \$
Sonnet z MCP		100%	0,078 \$
Opus bez		99%	0,547 \$
Opus z MCP		100%	0,182 \$

KOSZT

13× taniej

Najtańszy model z indeksem bije najdroższy bez

Haiku z MCP: 100% za 0,041 \$.
Opus bez MCP: 99% za 0,547 \$ — 13× drożej.

TRAFNOŚĆ

0 pominieć

Indeks zna każdy wiersz pod pełnym ID

Haiku bez MCP: „żaden plik nie wspomina ENT-Disposition.RW-001”. Encja zapisuje wiersz jako RW-001 ; indeks zna pełny ID.

CZAS

≤ 33 s

Przewidywalnie

Najdłuższe uruchomienie z MCP: 33 s. Bez: 153 s. W 33 z 36 par MCP szybszy i tańszy.

Ograniczenie: wzorcem był wynik indeksu, więc test mierzy szybkość i trafność znajdowania plików, nie jakość wzorców zapytań. Szczegóły: `scripts/knowledge-index/benchmark.md`.

Zmiana wymagania: stała kolejność

Relacje usuwasz na końcu, bo zapytanie po nich przechodzi.

- 1 Zapytaj o zasięg. `change_scope` dla `BFS-001.REQ-01` — w Warsztacie (tryb Zasięg) albo w terminalu. Wynik: 14 plików.
- 2 Wyeksportuj listę kontrolną do zgłoszenia. Link otwiera u kolegi ten sam widok.
- 3 Popraw treść w plikach z listy.
- 4 Dopiero teraz usuń relacje i wiersze, które mają zniknąć. Numer wiersza nie wraca do obiegu.
- 5 Sync i walidacja. `sync-relations --fix`, trzy walidatory.

```
$ node ../cli.js scope BFS-001.REQ-01
```

```
change_scope dla BFS-001.REQ-01:  
węzły 14, pliki 14.
```

```
.../scenarios/uc-001.md  
UC-001 przez:  
REQ-01 <-realizes- UC-001  
.../contracts/api-001.md  
API-001 przez:  
UC-001 -consumes-> API-001  
...
```

TERMINAL

CLI

`find`, `describe`, `scope`,
`impl`. Szukanie bez polskich
znaków: obsługa znajduje
„Obsługa dyspozycji”.

AGENT AI

MCP w Claude Code i Copilot

Pytasz zwykłym językiem:
„Zmieniam encję ENT-
Disposition, które pliki
przejrzeć?”. Agent pyta indeks
zamiast czytać foldery.

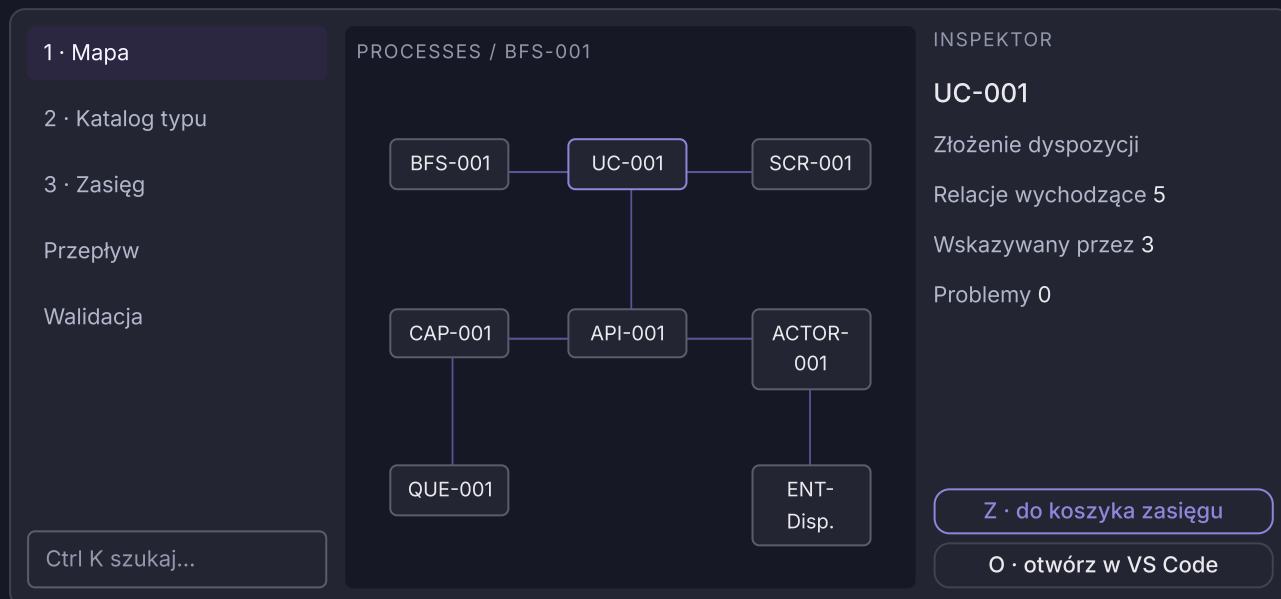
PRZEGLĄDARKA

Warsztat

Mapa, Zasięg, Przepływ,
walidacja — na tych samych
danych. Tylko czyta pliki; nie
da się nim zepsuć
dokumentacji.

Warsztat dokumentacji

Graf dokumentacji — docelowo kilku tysięcy dokumentów — jako obraz, nie lista plików. `npm install`, `npm run dev`, `http://localhost:5173`.



Schemat trybu Mapa: proces w kolumnach typów, Inspektor po prawej. Cały widok jest w adresie strony — link otwiera ten sam stan.

MAPA

Orientacja

Procesy i foldery wspólne; kolumny: mapy, ekrany, scenariusze, wymagania, odpowiedzialność, kontrakty, przepływy, dane.

ZASIĘG

Zakres zmiany

Koszyk węzłów, kolumny według kroków, podświetlona droga. Eksport listy kontrolnej. Symulacja usunięcia.

KATALOG TYPU

Kto czego używa

API według CAP, encje według LDM, kolumna „Używany przez”.

PRZEPŁYW

Architektura

Zdolności w kolumnach procesów; linie zdarzeń (QUE) i wywołań (API, INT). Klik linii: skąd kierunek, jakie encje płyną.

WALIDACJA

Porządkowanie

Trzy walidatory z filtrami; „Pokaż” przenosi na Mapę. Problem w podglądzie: linia, naprawa, „Kopiuje prompt dla agenta”.

PODGLĄD

Czytanie w kontekście

Markdown nad grafem, identyfikatory jako linki, diagramy Mermaid. Space podgląd, o VS Code, F dopasuj.

Czego narzędzia nie robią

Walidator sprawdza strukturę. Sens sprawdza analityk.

Nie oceniają sensu treści

Czy wymaganie jest poprawne, czy odpowiedzialność dobrze podzielona — to analityk.

Nie zgadną zależności, której nie napisałeś

Relacja powstaje z identyfikatora w treści. Jeśli API używa encji, trzeba to napisać.

`change_scope` to lista do przejrzenia

Zawęża przegląd do plików, które mogą wymagać zmiany. Nie mówi, co na pewno zmienić.

Aplikacja nie edytuje plików

Otwiera je w VS Code. Indeks zwraca pliki, nie treść.